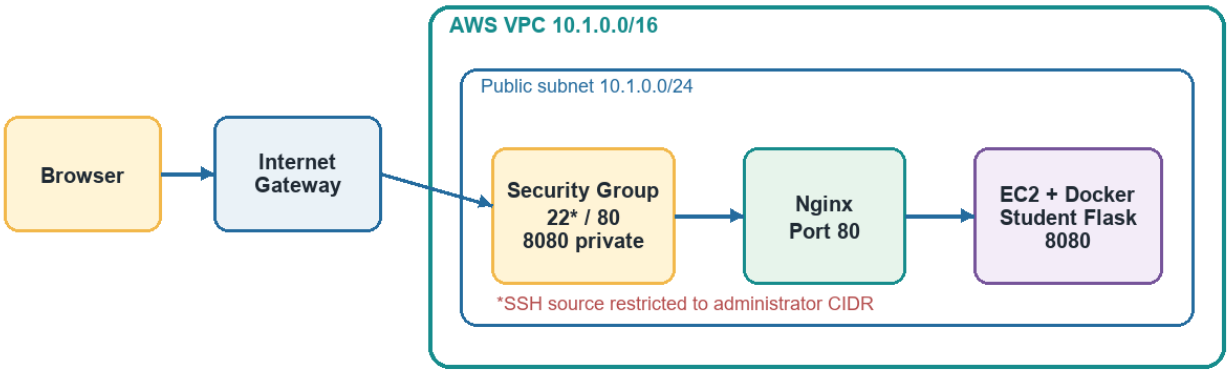
Infrastructure Automation Mini Project

Student Record and Academic Management Platform

Terraform and Ansible deployment on Amazon Web Services

Detailed mini project report

Prepared from the supplied Unit 4 assignment brief.



Project overview

This project explains how to automate a small academic records application on AWS. Terraform creates the cloud network and compute resources; Ansible configures the operating system and application; validation checks that the deployed service is reachable and healthy. The supplied brief uses the Enterprise Smart Library as a trainer example; this report focuses on the student implementation.

Learning outcomes: describe the architecture; organize Terraform and Ansible code; deploy a containerized Flask service; verify the deployment; explain security and environment choices.

Assignment deliverables

Deliverable	Weight	Evidence
Terraform VPC, subnet, security group, EC2 and S3	30%	Modular HCL, plan/apply, outputs and state approach
Ansible roles for system setup, Docker, Nginx and app	30%	Role tree, playbook and repeat-run evidence
End-to-end AWS deployment	20%	App reachable through public web endpoint
Validation and technical report	20%	Smoke-test logs, architecture diagram and repository link

1. Project scope and requirements

1.1 Problem statement

The Student Record and Academic Management Platform stores student profiles, course enrollment and academic marks. A Flask REST API is packaged as a Docker image. Automation should recreate cloud resources and host configuration from version-controlled definitions rather than manual console steps.

1.2 Goals and boundaries

- VPC: `10.1.0.0/16`; public subnet: `10.1.0.0/24`, matching the student row in the assignment.
- EC2 Linux host sized for the container and proxy. The brief gives `t2.micro` as a lab example, subject to region/account availability.
- Nginx is the public entry on port 80. The Flask service listens on port 8080 internally.
- Terraform provisions cloud resources; Ansible performs repeatable host configuration.
- Checks cover SSH readiness, services, container state and API responses.

This is a learning deployment, not a highly available production system. One instance and local SQLite are useful for demonstrating automation; real academic records require stronger privacy, recovery, availability and database controls.

1.3 Functional and non-functional requirements

Area	Requirement	Reason
Application	`GET /api/health`, `GET /api/students`, `POST /api/marks`	Check service and core workflows
Network	Public HTTP through Nginx; SSH restricted to administrator CIDR	Minimize exposed services
Compute	Ubuntu-based EC2 with container runtime	Run API and proxy
Repeatability	Terraform desired state and idempotent Ansible tasks	Rebuild and repair consistently
Data	Persistent path and tested backup/restore	Protect records from instance loss
Security	Least privilege, encryption, no secrets in source	Protect records and cloud account

2. Application and cloud architecture

The request path starts at a browser and reaches EC2 through the VPC internet gateway and security group. Nginx accepts HTTP on port 80 and forwards to the application container. Terraform provisions the network and instance; Ansible configures software after the host is reachable.

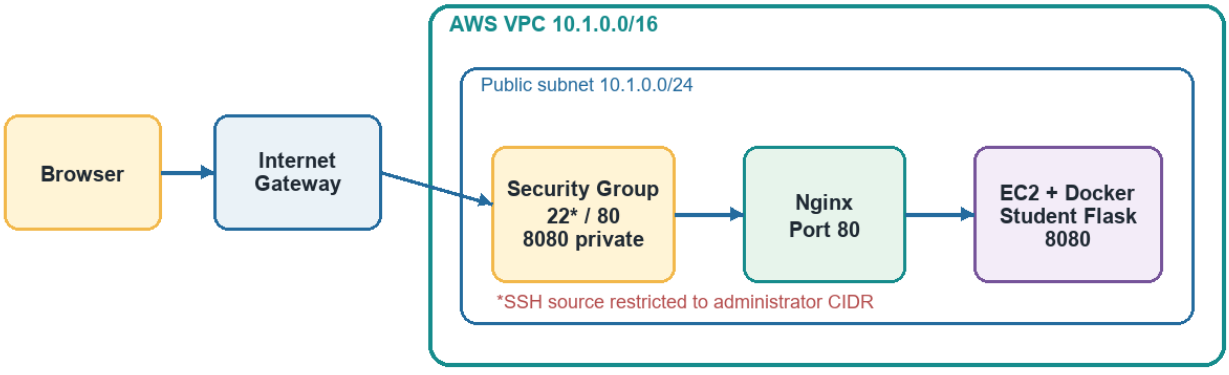


Figure 1. Student platform learning architecture. Allow SSH only from a known administrator range. Keep the application port private.

2.1 Component responsibilities

Component	Role in the project
VPC and subnet	Isolate resources; use `10.1.0.0/16` and `10.1.0.0/24`
Internet gateway and route	Connect public subnet to internet; default route points to IGW
Security group	Filter traffic: SSH restricted, HTTP public, app port internal
EC2	Linux host running Docker, Nginx and application container
Docker	Packages Flask application and dependencies; container port 8080
Nginx	Reverse proxy from public port 80 to application port 8080
S3	Private encrypted/versioned storage for backups or Terraform state

2.2 Request flow

1. A user requests the instance public DNS name or address on HTTP port 80.
2. The route table sends internet-bound subnet traffic through the attached gateway.
3. The security group permits intended inbound connections. Security groups are stateful.
4. Nginx proxies the request to the Flask container on its internal port.
5. The container reads or writes its persistent data and returns the response through Nginx.

Assignment interpretation: the trainer library example allows direct port 5000 access; the student project names port 8080. A safer design exposes only port 80 publicly and allows the app port only locally or from Nginx.

2.3 Trainer example and student target

Dimension	Library trainer	Student project
Domain	Books, borrowers, loans	Student records, enrollment, marks
VPC	`10.0.0.0/16`	`10.1.0.0/16`
App port	5000	8080
API checks	`/api/health`, `/api/books`, `POST /api/borrow`	`/api/health`, `/api/students`, `POST /api/marks`

3. Infrastructure as Code with Terraform

Infrastructure as Code describes resources in files and reconciles those declarations with cloud reality. Terraform uses HCL, a provider to communicate with AWS APIs, a dependency graph to order operations, and state to map configuration objects to real resources.

3.1 Core concepts

Concept	Meaning	Practical implication
HCL configuration	Readable resource and variable declarations	Keep inputs explicit and files understandable
Provider	AWS API integration	Pin a tested version; commit dependency lock file
State	Mapping between Terraform addresses and real resources	Treat as sensitive; restrict and version remote state
Plan	Proposed creates, updates and deletes	Review before applying; investigate replacement
Dependency graph	Ordering of related resources	Reference resource attributes rather than file order
Module	Reusable group of resources	Use where it improves reuse and clarity

3.2 Terraform lifecycle

Command	Purpose	Expected result
`terraform init`	Initialize providers and backend	Dependencies installed and lock file maintained
`terraform fmt -check`	Check formatting	Consistent HCL
`terraform validate`	Check structure and types	Configuration is syntactically valid
`terraform plan`	Compare desired and actual state	Reviewable execution plan
`terraform apply`	Apply approved changes	Resources created/changed and state updated
`terraform destroy`	Remove managed resources	Use only after verifying target and retained data

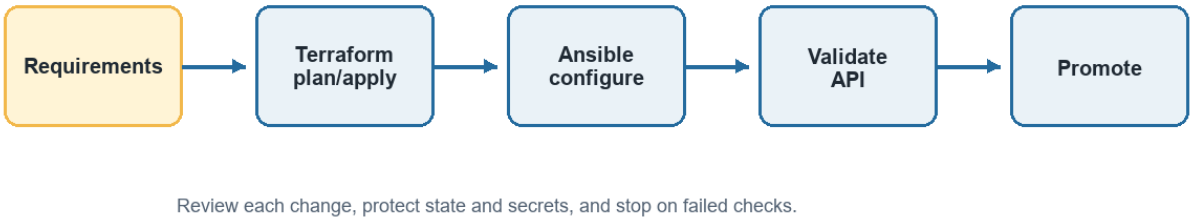


Figure 2. Review changes before applying; validate before promoting.

3.3 Suggested Terraform layout

```
terraform/
  versions.tf  providers.tf  variables.tf
  network.tf   security.tf   compute.tf
  storage.tf   outputs.tf
  terraform.tfvars.example
  modules/ (optional)
```

Illustrative variable-driven network fragment:

```
variable "project_name" { type = string }
variable "vpc_cidr" { type = string, default = "10.1.0.0/16" }
variable "subnet_cidr" { type = string, default = "10.1.0.0/24" }

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = { Name = "${var.project_name}-vpc" }
}
```

A complete network also needs an internet gateway, route table, subnet association and tags. Define narrow, explicit security group rules. Do not commit SSH keys, cloud secrets or unreviewed tfvars files. Use the provider credential chain or approved role.

3.4 State and collaboration

Local state can suit a short individual exercise. Team use needs remote state with access control, encryption, versioning, backups and locking. The assignment describes S3 and DynamoDB locking. Current Terraform S3 backend docs support S3 lock files and mark DynamoDB locking deprecated; follow the instructor's requirement and the lab's Terraform version. [1]

State is sensitive. It may contain resource attributes and secrets. Restrict read access, enable bucket versioning and block public access. Never store student records in Terraform state.

4. Configuration management with Ansible

Terraform creates the cloud foundation. Ansible configures the operating system and software. Ansible commonly connects over SSH without a proprietary target agent. Most modules are idempotent: if desired state already exists, another run makes no change. [3]

4.1 Role design

Role	Typical tasks	Validation
`common`	Package metadata, utilities, app user and directories	Packages and ownership correct
`docker`	Install Docker from approved source; enable service	Docker active; permissions correct
`nginx`	Install proxy, deploy template, reload on change	Config valid; port 80 responds
`student_app`	Pull pinned image, run container and mount data	Container healthy; API responds

4.2 Suggested structure

```
ansible/
  ansible.cfg  inventory/hosts.ini
  playbooks/site.yml
  group_vars/{all,dev,staging,prod}.yml
  roles/{common,docker,nginx,student_app}/
    tasks/ handlers/ templates/ defaults/
```

Example playbook:

```
- name: Configure student platform hosts
  hosts: student_servers
  become: true
  roles:
    - common
    - docker
    - nginx
    - student_app
```

Use Ansible modules such as package, service, template, user and file instead of shell commands where possible. Use handlers to validate and reload Nginx only when its configuration changes. For shell tasks, specify when they run and how Ansible decides whether they changed state.

4.3 Demonstrating idempotency

- Run the playbook twice in a test environment; explain any changes on the second run.
- Pin the image tag or digest instead of using `latest`.
- Use a handler for changed Nginx configuration.
- Use check mode where supported, but do not assume every task supports it fully.
- Save the play recap and explain changed or failed tasks.

5. Terraform–Ansible orchestration

Terraform assigns the host address; Ansible needs an inventory. Exporting the address as a Terraform output and generating inventory dynamically avoids copy-and-paste errors.



Figure 3. Terraform provisions first; Ansible configures after SSH becomes reachable.

5.1 Handoff sequence

6. Terraform creates network, security group and instance.
7. Terraform exposes the instance public address or DNS name.
8. A wrapper reads the output and renders an Ansible inventory entry.
9. Automation waits for SSH readiness.
10. Ansible installs services and deploys the image.
11. Smoke tests verify Nginx and API health before promotion.

Example output and inventory:

```
output "student_server_public_ip" {
  value = aws_instance.student.public_ip
}
[student_servers]
student-1 ansible_host=203.0.113.10 ansible_user=ubuntu

[student_servers:vars]
ansible_python_interpreter=/usr/bin/python3
```

The IP is reserved for examples; replace it with the Terraform output. Do not put private key material in inventory.

5.2 Orchestration sequence

```
terraform init
terraform validate
terraform plan -out=tfplan
# Review the plan before applying
terraform apply tfplan
terraform output -raw student_server_public_ip
# Render inventory and wait for SSH, then:
ansible-playbook -i inventory/hosts.ini playbooks/site.yml
```

If SSH is unreachable, check the route table, public address, SSH source CIDR, key/user and instance readiness. Do not let retries hide an authentication or network failure.

6. Environments and promotion

Environment parity means the same application and configuration behavior is tested in development, staging and production. Differences should be explicit inputs, not undocumented manual edits.

Environment	Purpose	Controls
Development	Fast, low-cost iteration	Small instance, synthetic data, short-lived resources

Staging	Production-like integration and release checks	Matching software, access controls, representative tests
Production	Serve users and protect records	High availability, strict IAM, encryption, monitoring, recovery tests

For a lab, separate variable files and Ansible group variables are understandable. Terraform workspaces are another option, but are not a security boundary. Larger systems often benefit from separate state and accounts. Promote the same immutable image through stages rather than rebuilding a floating tag each time.

7. Validation, security and operations

A successful Terraform apply does not prove the application works. Check network reachability, services, container state, persistent storage and application behavior.

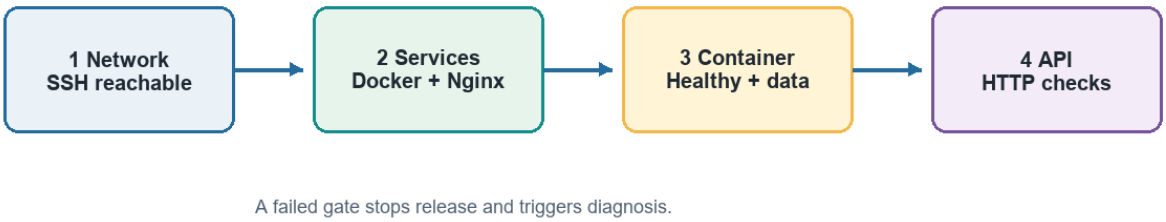


Figure 4. Any failed gate should stop promotion and produce a diagnostic.

7.1 Validation matrix

Gate	Check	Example
Network and OS	SSH and OS readiness	`ssh ubuntu@$IP uptime` from approved network
Services and ports	Docker and Nginx active	Ansible service checks; `curl -I http://$IP/`
Container and data	Correct image, running container, writable mount	`docker ps`; inspect logs and mount permissions
API smoke tests	Health, student and marks routes	`curl -fsS http://$IP/api/health`; test GET and POST

The POST test depends on the application schema. Use synthetic records, assert status and response fields, and avoid printing personal information into CI logs.

7.2 Security controls

- Restrict SSH to a known administrator CIDR or use managed access. Do not leave port 22 open to `0.0.0.0/0`; AWS Config flags unrestricted SSH. [2]
- Expose only Nginx publicly. Keep the Flask container port internal or bind it to localhost.
- Give IAM roles only required permissions. Never place access keys in source, images, user data or logs.
- Enable S3 Block Public Access, encryption, versioning and restrictive policies. Separate Terraform state and backup objects where appropriate.
- Use HTTPS for real user traffic. Public HTTP-only access is unsuitable for sensitive student records.
- Use synthetic data in development and screenshots; follow institutional privacy rules.
- Patch host and images, pin dependencies and retain logs without exposing secrets.

7.3 Backups and recovery

A backup is useful only if restoration is tested. Document the data set, frequency, retention, access and restore steps. SQLite on one instance is fragile for concurrent production use; a managed database with automated backups is a scale-up path. Terraform state and application backups have different access and retention needs.

7.4 Troubleshooting

Symptom	Likely cause	First checks
SSH timeout	No public route, wrong CIDR or missing public IP	IGW, subnet route, route table and security group
SSH denied	Wrong username/key	Ubuntu login, selected key, protected key path
Nginx 502	App stopped or upstream port mismatch	Container logs, port mapping and proxy target
HTTP 404	Route or proxy path mismatch	Flask routes and Nginx location
Terraform replacement	Immutable change or wrong state	Read plan; confirm backend/workspace
Ansible repeats changes	Non-idempotent shell or unstable input	Prefer stateful module; inspect changed tasks

8. Student implementation blueprint

8.1 Build order

12. Confirm the app runs locally; document port, health endpoint, variables and data path.
13. Create and review Terraform network resources and least-exposure security rules.
14. Create the instance and outputs; verify approved SSH access.
15. Implement Ansible roles: common, Docker, Nginx and student app.
16. Automate output-to-inventory handoff and wait for SSH.
17. Run all validation gates and run Ansible again to demonstrate idempotency.
18. Capture sanitized logs, diagrams and explanations; clean up lab resources after confirming data retention.

8.2 Repository structure

```
student-platform-infra/
  README.md  terraform/  ansible/  app/
  scripts/  docs/  .gitignore
```

8.3 Smoke-test outline

```
#!/usr/bin/env bash
set -euo pipefail
BASE_URL="${1:?usage: smoke_test.sh http://host}"
curl --fail --silent --show-error "$BASE_URL/api/health"
curl --fail --silent --show-error "$BASE_URL/api/students"
# POST a synthetic marks record using the actual API schema
```

8.4 Evidence package

- Architecture diagram showing network boundaries, traffic direction and ports.
- Sanitized Terraform plan/apply output.
- Ansible output, including second run and idempotency evidence.
- Smoke tests for health, students and marks endpoints.
- Security, limitations, backup and cleanup explanation.
- The actual repository URL requested in the assignment; do not invent a placeholder.

9. Evaluation checklist and conclusion

Rubric area	Before submission
Terraform 30%	VPC, subnet, SG, EC2 and S3 are modular and parameterized; plan reviewed; outputs and state explained
Ansible 30%	Roles cover system setup, Docker, Nginx and app; repeat run explained
Deployment 20%	App responds through Nginx on port 80; internal port is not needlessly public
Report 20%	Four validation layers, sanitized logs, diagram and repository URL included

Terraform establishes the cloud foundation; Ansible configures the host. A dynamic handoff joins them, while validation gates confirm the service before promotion. Reproducibility depends on versioned configuration, protected state, pinned images, idempotent tasks and controlled access.

Key principle: automate desired state, review changes before applying them, and verify the running application from the user's point of view.

Appendix A. Command reference

Purpose	Command
Initialize	`terraform init`
Format and validate	`terraform fmt -check && terraform validate`
Preview changes	`terraform plan`
Apply reviewed changes	`terraform apply`
Inspect outputs	`terraform output`
Run Ansible	`ansible-playbook -i inventory/hosts.ini playbooks/site.yml`
Check connectivity	`ansible -i inventory/hosts.ini student_servers -m ping`
Check API	`curl -fsS http://PUBLIC_HOST/api/health`
Remove lab resources	`terraform destroy` only after checking environment and data retention

References

19. [HashiCorp Terraform S3 backend documentation](https://developer.hashicorp.com/terraform/language/backend/s3) — state versioning and locking options.
20. [AWS Config restricted-SSH rule](https://docs.aws.amazon.com/config/latest/developerguide/restricted-ssh.html).
21. [Ansible playbooks introduction](https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html) — desired state and idempotency.
22. [AWS EC2 security group documentation](https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/creating-security-group.html).

Source brief: Assignment_5340_Content_Document_20261009101309920AM.pdf, Unit 4 Infrastructure Automation (AWS), pages 1–9. Assignment requirements are treated as project instructions; added notes explain security and learning-design limitations.